This isn't necessarily bad, but handling errors becomes cumbersome. We would check each line against possible error responses and handle an error appropriately. With statements make this easier for us to implement.

A with statement consists of clause lines, followed by a main body, followed optionally by else clauses. Each clause line (pattern <- term) is evaluated top to bottom until all are evaluated, or until one of the lines doesn't match. Once all lines are matched, the main body is executed and its value is returned.

If a line doesn't match and there's an else body, then the else is evaluated like a case statement is. It pattern-matches against the error value based on your provided error handling code. You use this opportunity to gracefully handle the error—log it out, throw an exception, return an error tuple, and so on. If there isn't an else body, then the result that didn't match is directly returned.

Here's an example of multiple steps coming together to produce a result. Any of these steps could fail, and we want to gracefully handle the result:

```
$ iex
iex>
with {:ok, files} <- File.ls("."),
     [first_file | _rest] <- Enum.sort(files),
     {:ok, %{ctime: created}} <- File.lstat(first_file) do
  IO.puts("The file #{first_file} was created at #{inspect created}")
else
  {:error, _} -> {:error, "file system failed"}
  [] -> {:error, "no files"}
end

# Your result will vary
The file .DS_Store was created at {{2022, 9, 21}, {20, 43, 29}}
```

One thing that trips up new Elixirists is using = instead of <- in a with statement. This will compile and even run, but any failure that would normally have triggered the else clause will instead raise a MatchError. If you accidentally do this, you may be unpleasantly surprised because the with statement loses its error-handling ability.

What happens if we run this code in a directory that doesn't have any files in it? Create a new empty directory and start iex in it:

```
$ mkdir empty && cd empty
$ iex
iex> # same code from previous example
{:error, "no files"}
```